

第10章 中间代码生成 2026

赵银亮

本章的知识图谱

- IR:(高级IR:(AST QAST) CFG型IR:(LLVM-IR:(CFG SSA) 三地址码:(TAC QTAC P-code 四元式)) 字节码 专用IR:(MLIR XLA-IR Torch IR))
- 中间代码生成:(表达式:(算术表达式 条件表达式/布尔表达式) 语句:(表达式语句 赋值语句 控制语句:(if while goto) 函数调用语句))

- 实用函数:
  - newl()、gen[]、merg-code()、genl[]
  - genq()、emit()、emit[]

第10章 中间代码生成

- 使用符号表将执行语句翻译为QTAC代码
- 关于中间语言：
  - 工业编译器：LLVM-IR（常用）、GCC-RTL
  - 经典：TAC（QTAC）、P-code（逆波兰、栈式抽象机、由Niklaus Wirth设计）、AST
  - AI编译器：MLIR（让不同层级的 IR 高效协作，覆盖控制流、张量、硬件等）

方言	层级	用途
Linalg	高层	线性代数 / 张量计算，保留模型语义（如卷积、矩阵乘）
Tensor	高层	张量形状、类型、布局描述
GPU	中层	GPU 并行模型（线程块、共享内存）
SPIR-V	中层	异构硬件指令集映射（GPU/NPU）
LLVM	低层	对接 LLVM IR，生成 CPU/GPU 机器码
Triton	中层	高性能 GPU 算子（如矩阵乘、FlashAttention）

10.1 中间语言

- 目标定位：机器无关性、高级语言无关性、兼顾可分析性与可优化性
- 语义与表达能力：语义完备性（变量、常量、分支、循环、函数调用、数组 / 结构体、指针、地址操作、全局 / 局部作用域、异常、虚函数等）；类型系统（带类型标注、支持基础类型、聚合类型、指针、函数类型、别名）；CFG）
- 树形 IR（AST）、三地址码 / 四元式 IR（TAC、QTAC）、线性指令式 IR、图 IR（CFG +SSA）等等
- 内存模型与数据布局、容易把高级语言翻译为它；
  - 容易把它翻译为机器语言；
  - 容易进行代码优化。

典型中间语言

- Linear IR
  - 逆波兰表示：如将 E op T 翻译为 E' T' op
  - TAC/四元式
- AST/DAG
- SSA+CFG

例：三地址代码

```
int x;
int fact(int n; int a;){
    if (n<=1) return a
    else return fact (n-1, n*a,)
};
x=123+fact(5,1,);
print x
```

注：中间代码生成的目标是生成函数的代码，其中用到各个语法单元的代码生成功能。

```
@code=[
    t4=123; t5=5; t6=1;
    PAR t6; PAR t5;
    t7=CALL fact, 2;
    x=t4+t7;
    PRINT x]
```

```
fact@code=[
    IF n==1 THEN l1 ELSE l2;
    LABEL l1; RETURN a;
    GOTO l3;
    LABEL l2;
    t1=n-1; t2=n*a;
    PAR t2; PAR t1;
    t3=CALL fact, 2;
    RETURN t3;
    LABEL l3]
```

QTAC

- ⟨QTAC-program⟩⟶⟨QTAC-global-list⟩⟨QTAC-func-list⟩⟨QTAC-ins-list⟩
- ⟨QTAC-global-list⟩⟶ε | ⟨QTAC-global⟩⟨QTAC-global-list⟩
- ⟨QTAC-global⟩⟶`ik`d`;`
- ⟨QTAC-func-list⟩⟶ε | ⟨QTAC-func⟩⟨QTAC-func-list⟩
- ⟨QTAC-func⟩⟶`define`d`(`a_1, …, a_n`){LABEL`d`;` ⟨QTAC-ins-list⟩`}` \\$0 \leqslant n \leqslant 8$
- ⟨QTAC-ins-list⟩⟶⟨QTAC-ins⟩;|⟨QTAC-ins-list⟩⟨QTAC-ins⟩;

QTAC（续）

- ⟨QTAC-ins⟩⟶LABEL l | GOTO e | IF q ⟨rop⟩ a THEN l ELSE l
- q⟶d | ⟨temp⟩
- a⟶q | k | l
- b⟶q | k
- e⟶q | l
- ⟨QTAC-ins⟩⟶PAR a | q = CALL d, k | RETURN a
- ⟨QTAC-ins⟩⟶$q = q$ ⟨aop⟩ a | $q = a$
- ⟨QTAC-ins⟩⟶$q = M[q]$ | $M[q] = a$
- ⟨TAC-ins⟩⟶NOP | q = CONVERT q, t

LLVM-IR

- Clang 默认生成的原始 IR
  - 所有局部变量都用 alloca 放在栈上
  - 用 load/store 读写变量
  - 可以反复赋值
  - 不是 SSA（% 开头的标识符只能被赋值 / 定义 恰好一次）
- 经过 LLVM 优化器的 mem2reg 优化 后成为 SSA 形式
  - 去掉 alloca，去掉针对栈变量的 load / store
  - 每个 %var 只定义一次
  - 循环用phi 节点迭代语法：%val = phi i32 [%val1, %block1], [%val2, %block2]含义：若控制流来自 %block1，则 %val 取值 %val1；若来自 %block2，则取 %val2。在循环中特别处理。
  - 全局数组读写的 load/store 合法保留

```
int a[10]; int b[5][20]; int c[10][5][20];
c[i][j][k]=a[x]+b[y][z];
```

- LLVM_IR:
- @a = global [10 x i32] zeroinitializer
- @b = global [5 x [20 x i32]] zeroinitializer
- @c = global [10 x [5 x [20 x i32]]] zeroinitializer
- %addr_a = getelementptr [10 x i32], [10 x i32]* @a, i32 0, i32 %x
- %val_a = load i32, i32* %addr_a
- %addr_b = getelementptr [5 x [20 x i32]], [5 x [20 x i32]]* @b, i32 0, i32 %y, i32 %z
- %val_b = load i32, i32* %addr_b
- %sum = add i32 %val_a, %val_b
- %addr_c = getelementptr [10 x [5 x [20 x i32]]], [10 x [5 x [20 x i32]]]* @c, i32 0, i32 %i, i32 %j, i32 %k
- store i32 %sum, i32* %addr_c
- %指针 = getelementptr 元素类型, 元素类型* %基地址, i32 索引1, i32 索引2...

```
QTAC:
i80 a;
i800 b;
i8000 c;
...
t1=x*4;
t2=a+t1;
t3=M[t2];
t4=y*20;
t5=t4+z;
t6=t5*4;
t7=M[t6];
t8=t3+t7;
t9=i*5;
t10=t9+j;
t11=t10*20;
t12=t11+k;
t13=t12*4;
M[t13]=t8;
```

- 对照qsort程序的LLVM-IR与LLVM SSA+CFG表示
- 可用AI生成，然后进行观察对比SSA方面的处理

三地址代码实现为四元式

- 三地址代码

$$q = q \langle \text{aop} \rangle a$$

- 四元式：$k\,(\langle \text{aop} \rangle, q_1, a, q_2)$
  - k是四元式的编号，实现为存储单元地址，连续编号；
  - $\langle \text{aop} \rangle$二元(也可一元或零元)运算符；
  - 第1、第2操作数和结果变量依次为第2-4元(可以缺省)；
  - IF指令特殊处理。

例：从三地址代码到四元式

```
[INPUT x
t1 = 0
IF t1 < x THEN l1 ELSE l2
LABEL l1
fact = 1
GOTO l3
LABEL l2
fact = 2
LABEL l3
fact = fact * x
x = x – 1
IF x == 0 THEN l4 ELSE l3
LABEL l4
PRINT fact]
```

```
100 (INPUT, x, _, _)
101 (J<, 0, x, 103)
102 (J, _, _, 105)
103 (=, 1, _, fact)
104 (J, _, _, 106)
105 (=, 2, _, fact)
106 (*, fact, x, fact)
107 (-, x, 1, x)
108 (J=, x, 0, 110)
109 (J, _, _, 106)
110 (PRINT, fact, _, _)
```

四元式适用于增量式代码生成模式，三地址代码除此还适合代码作为属性值的生成模式。

基于图的中间表示

- 抽象语法树 （AST）
  - TranslationUnitDecl
    - FunctionDecl "foo" (int (int))
    - ParmVarDecl "a" (int)
    - CompoundStmt
      - ReturnStmt
        - BinaryOperator '+' (int)
          - DeclRefExpr "a" (int)
          - IntegerLiteral 1 (int)

- ANTLR：跨语言解析器生成器，**自动构建AST**并支持树遍历
  - 支持多语言（Java/C++/Python 等），广泛用于编译器 / 静态分析。对接 Clang CodeGen 生成 LLVM IR。
- 有向无循环图(Directed Acyclic Graph，简称DAG)
  - 在一个DAG中代表公共子表达式的结点具有多个父结点

QAST

- 各语言 / 工具链通常定义自己的 AST 格式，互不兼容但设计思想相近（具体问AI）
- QAST是针对QL文法的AST设计结果
- QL文法与主文法的主要区别：
  - 是否允许嵌套函数声明，前者不允许后者允许。
  - 数组的语法：声明与引用分离/共享
- QAST带有扁平化AST（具体问AI）一些特点
- QAST同样适用于主文法，并能够翻译为ARM代码

QAST

- $P \longrightarrow B$

- NodeP:(left: NodeDList right:NodeDlist next: NodeSList)
- NodeB:(left:NodeDList right:NodeSList)
-

- $T \longrightarrow$ int | void

- NodeTPrim:(name:int/void value:NUM/VOID)
- NodeTArray:(left:NodeE right:NodeTArray/NULL)
- //维长表达式、数组类型或NULL，NULL表示是最后一维

- $D \longrightarrow TV \mid HB$
- $H \longrightarrow T\boldsymbol{d}(\check{A})$
- $B \longrightarrow \{\check{D}\check{S}\}$

- NodeDVar:(name:d.name left:NodeT right:NodeTArray/NULL)
- //数组名、元素类型、数组类型，NULL表示简单变量
- NodeDFunc:(name:d.name left:NodeT right:NodeAList next:NodeB)
- //函数名、返回类型、形参表、体块

- $S \longrightarrow ; \mid E; \mid \text{if}(C)S \mid \text{if}(C)S \text{ else } S \mid \text{while}(C)S \mid \text{return } E; \mid B$

- NodeNOP:(name:NOP)
- NodeSExpr:(left:NodeE)
- NodeIF:(left:NodeC right:NodeS NodeS)
- NodeWHILE:(left:NodeC right:NodeS)
- NodeRETURN:(left:NodeE)

- $V \longrightarrow \boldsymbol{d} \mid V[E]$
- $E \longrightarrow \boldsymbol{i} \mid V \mid \boldsymbol{d}(\check{R}) \mid \boldsymbol{u}E \mid E\boldsymbol{o}E \mid (E)$

- NodeNUM:(value:i.value) //[0-9]+
- NodeID:(name:d.name) //[a-zA-Z]+
- NodeArrayAccess:(left:NodeArrayAccess/NodeID right:NodeE)
- NodeCall:(name:d.name right:NodeRList)
- NodeUOP:(name:u.name left:NodeE) //UOP={−}
- NodeAOP:(name:o.name left:NodeE right:NodeE) //AOP={+,−,×,/,=,+=,=+}

- $\check{D} \longrightarrow \varepsilon \mid \check{D}D;$
- $\check{A} \longrightarrow \varepsilon \mid \check{A}A;$
- $\check{S} \longrightarrow S \mid \check{S};S$
- $\check{R} \longrightarrow \varepsilon \mid \check{R}E,$

- NodeDList:(left:NodeD right next:NodeDList)
- NodeAList:(left:NodeA right: next:NodeAList)
- NodeSList:(left:NodeS right next:NodeSList)
- NodeRList:(left:NodeR right next:NodeRList)

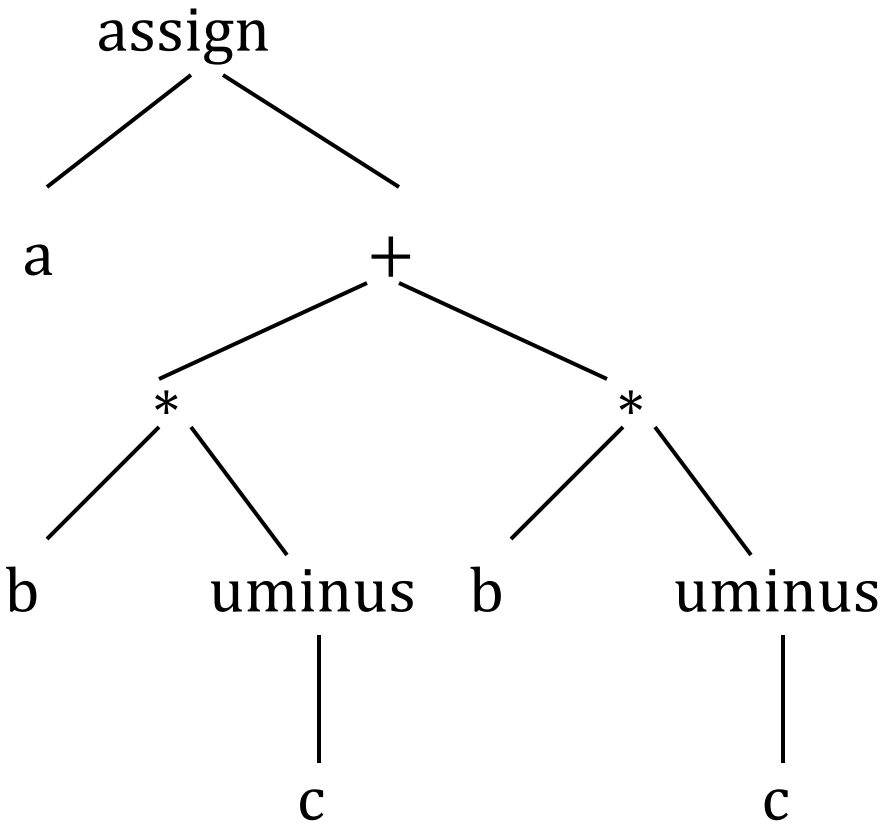
- $A \longrightarrow T\boldsymbol{d} \mid T\boldsymbol{d}[]$

- NodeAFunc:(name:d.name left:T.type right:NodeTList)
- NodeAVar:(name:d.name left: NodeT) //简单变量和数组
-

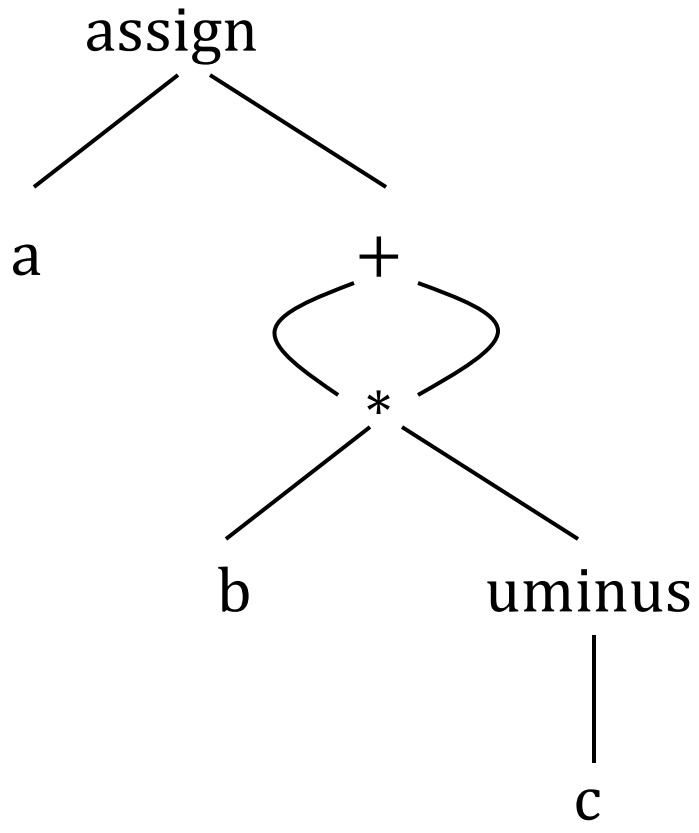
- $C \longrightarrow E \mid E\boldsymbol{r}E \mid !C \mid C\&\&C \mid C||C \mid (C)$

- NodeNZ:(name:NZ left:NodeE)
- NodeROP:(name:r.name left:NodeE right:NodeE)
- NodeUOP:(name:! left:NodeC)
- NodeAND:(name:&& left:NodeC right:NodeC)
- NodeOR:(name:|| left: :NodeC right:NodeC)

例：AST与DAG

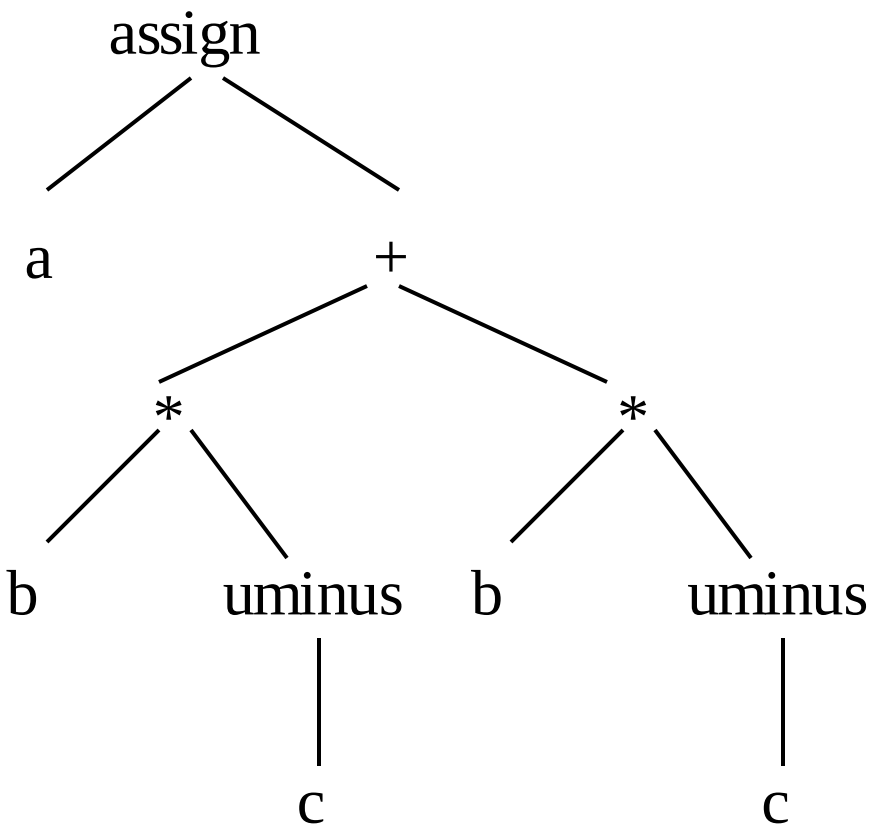


抽象语法树



DAG

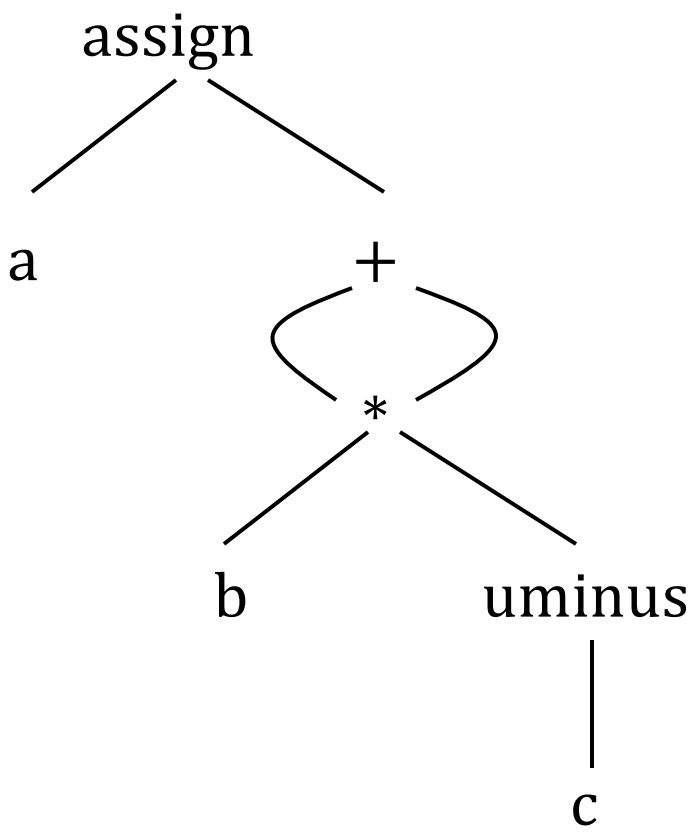
例：AST与三地址代码



抽象语法树

[t1=-c
t2=b*t1
t3=-c
t4=b*t3
t5=t2+t4
a=t5]

例：DAG与三地址代码



DAG

[t1=-c
t2=b*t1
t3=t2+t2
a=t3]

10.2 简单算术表达式的翻译

- 不含数组引用和函数调用的算术表达式
  - E⟶i | d | E op E | - E | (E)
- 产生代码实现表达式的计算过程，这些代码在编译时生成，在运行时执行。当执行时得到表达式的值。

- 设计属性名
  - code用于存放由表达式翻译产生的中间代码，
  - place中存放一个变量，这个变量存放代码的执行结果

- 例：若E.code=[t1=24;t2=t1*36]那么E.place='t2
- 注：'x表示 求值结果为x；而x求值结果为x的值。
- 通常用'x'或"x"，表示字符x，但差别在于字符和符号不同。

简单算术表达式

- E⟶i | d | E op E
- E⟶i {
  t=newvar();
  E.place=t;
  E.code=gen[?t = ?getv(i)]}
- E⟶d {
  x=getn(d);
  if(lookup(x)==UNBOUND)error();
  E.place=x;
  E.code=[]}
- E⟶E op E {
  E[0].place=newvar();
  E[0].code=E[1].code++E[2].code++
  gen[?E[0].place = ?E[1].place ?getn(op) ?E[2].place]}

简单算术表达式

- E⟶(E) | -E

- E⟶(E)　　{
  E[0].place=E[1].place;
  E[0].code=E[1].code }

- E⟶-E　　{
  E[0].place=newvar();
  E[0].code=E[1].code++
  gen[?E[0].place = - ?E[1].place] }

例：句子的带注释的规范归约

- 算术表达式a*(x-2)+y的LR制导的翻译

a*(x-2)+y⇨E*(x-2)+y　　E.place=a; E.code=[]

⇨E*(E-2)+y　　E[2].place=x; E[2].code=[]

⇨E*(E-E)+y　　E[3].place=t1; E[3].code=[t1=2]

⇨E*(E)+y　　E[2].place=t2; E[2].code=[t1=2;t2=x-t1]

⇨E*E+y　　E[2].place=t2; E[2].code=[t1=2;t2=x-t1]

⇨E+y　　E.place=t3; E.code=[t1=2;t2=x-t1;t3=a*t2]

⇨E+E　　E[2].place=y; E[2].code=[]

⇨E　　E.place=t4; E.code=[t1=2;t2=x-t1;t3=a*t2;t4=t3+y]

注：句子a*(x-2)+y带注释的规范归约如上所示。每一行的注释部分为这步归约时进行求值的各属性及求值结果。其中变元的索引值表示对应句型中该变元的出现，从1开始算起。如第三行句型E*(E-E)+y的标注中使用E[3]指第三个E。对第一、第二个E没有标注是因为分别在第一、第二行标注过了，期间没有变化。其余类推。在标注时E.place属性标注为变量名是最终结果，没有带‘。

E⟶i {t=newvar();E.place=t;
E.code=gen[?t = ?getv(i)]}

E⟶d {x=getn(d);
if(lookup(x)==UNBOUND)error();
E.place=x; E.code=[]}

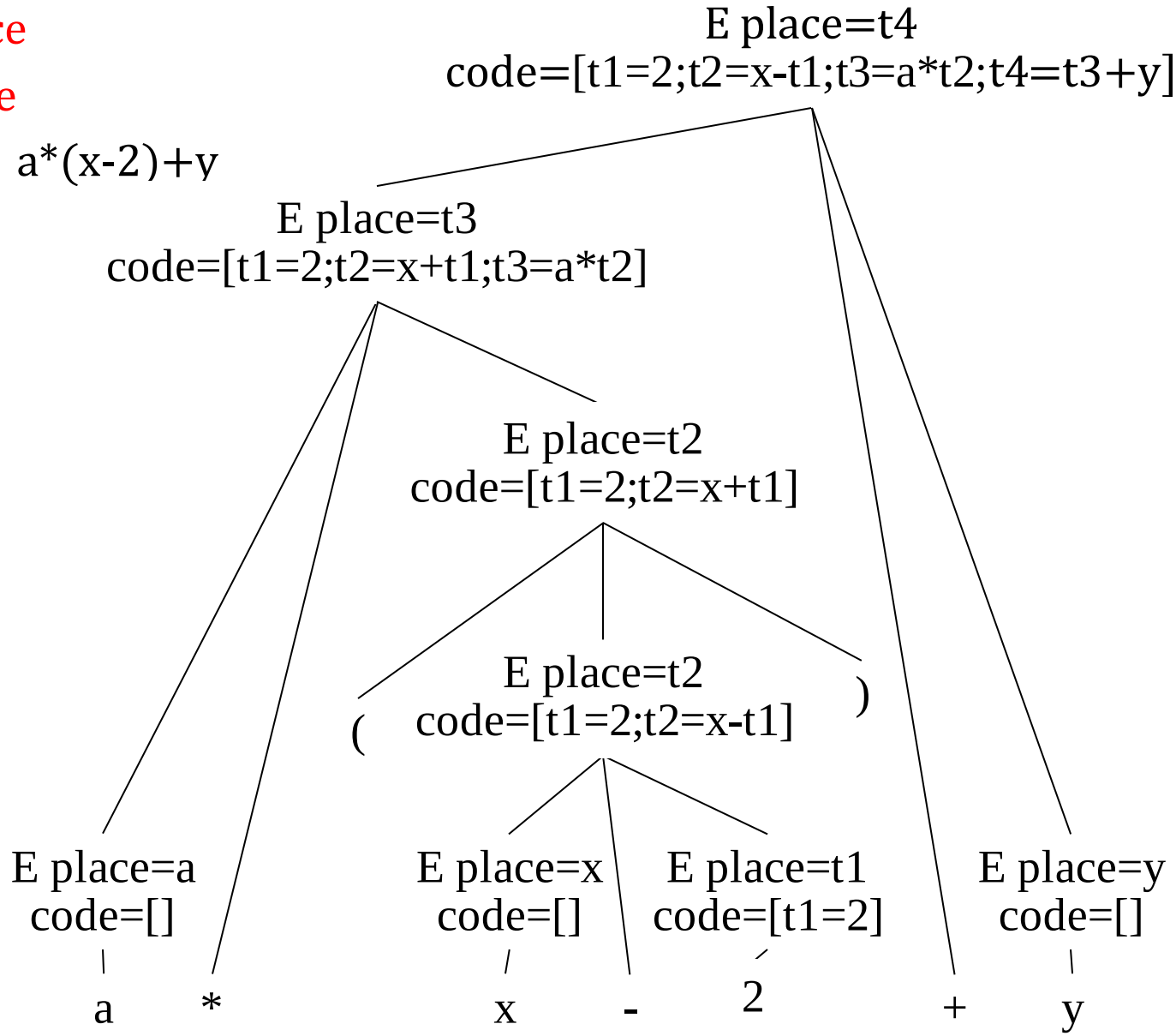
E⟶E op E {E[0].place=newvar();
E[0].code=E[1].code++E[2].code++
gen[?E[0].place = ?E[1].place
?getn(op) ?E[2].place]}

E⟶(E) {E[0].place=E[1].place;
E[0].code=E[1].code }

符号栈	place栈	code栈	w	action
#	#	#	a*(x-2)+y#	s
#a	#-	#-	*(x-2)+y#	r
#E	#a	#[]	*(x-2)+y#	s
#E*	#a-	#[]-	(x-2)+y#	s
#E*(	#a--	#[]--	x-2)+y#	s
#E*(x	#a--	#[]---	-2)+y#	r
#E*(E	#a--x	#[]--[]	-2)+y#	s
#E*(E-	#a--x-	#[]--[]-	2)+y#	s
#E*(E-2	#a--x--	#[]--[]--	)+y#	r
#E*(E-E	#a--x-t_1	#[]--[]-[t_1=2]	)+y#	r
#E*(E	#a--t_2	#[]--[t_1=2; t_2=x-t_1]	)+y#	s
#E*(E)	#a--t_2-	#[]--[t_1=2; t_2=x-t_1]-	+y#	r
#E*E	#a-t_2	#[]-[t_1=2; t_2=x-t_1]	+y#	r
#E	#t_3	#[t_1=2; t_2=x-t_1; t_3=a*t_2]	+y#	s
#E+	#t_3-	#[t_1=2; t_2=x-t_1; t_3=a*t_2]-	y#	s
#E+y	#t_3--	#[t_1=2; t_2=x-t_1; t_3=a*t_2]--	#	r
#E+E	#t_3-y	#[t_1=2; t_2=x-t_1; t_3=a*t_2]-[]	#	r
#E	#t_4	#[t_1=2; t_2=x-t_1; t_3=a*t_2; t_4=t_3+y]	#	acc

带注释语法树

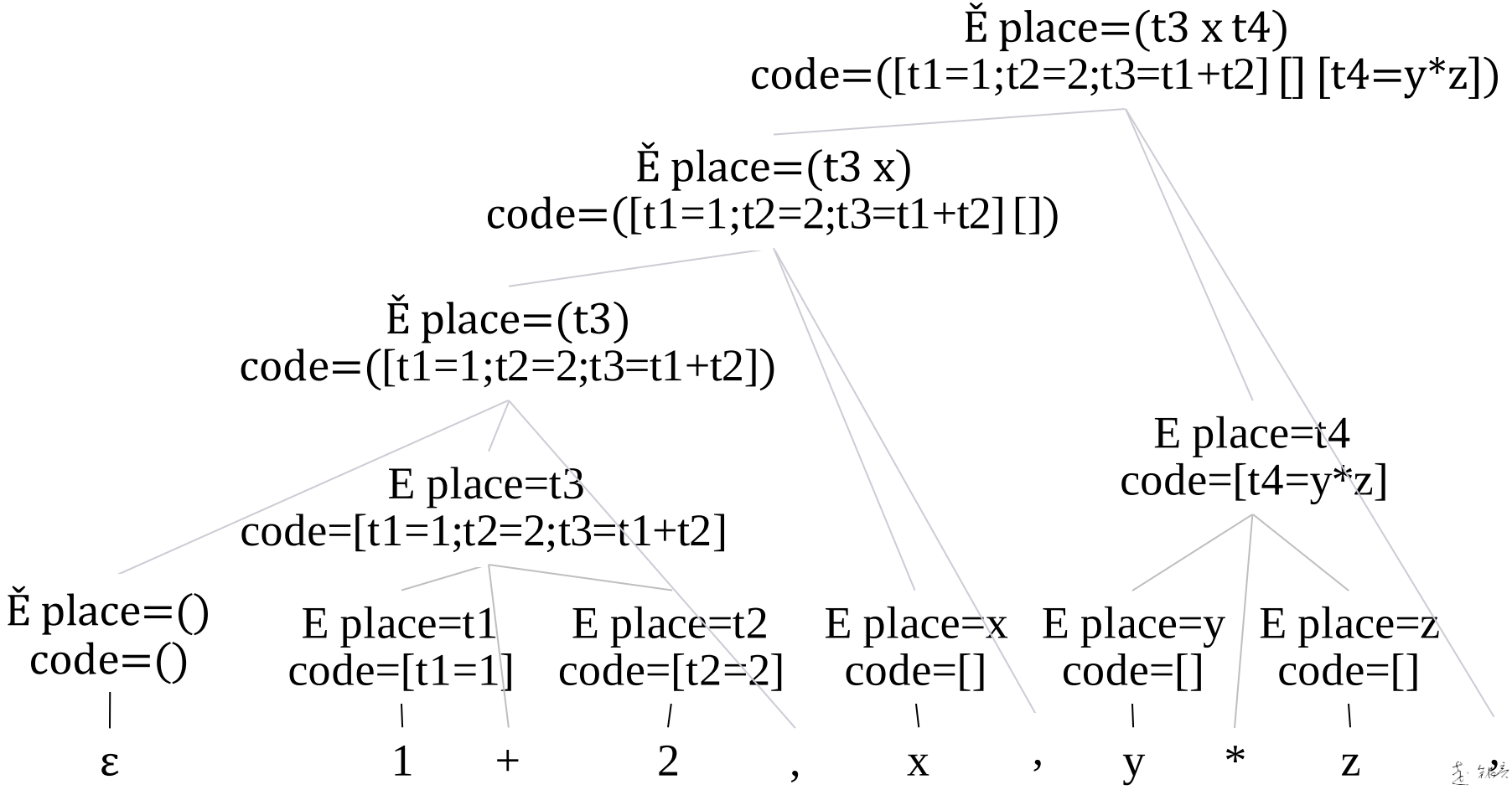
- 属性名：place
- 属性名：code
- 算术表达式：a*(x-2)+y



表达式列表

Ě⟶ε {Ě.place=NIL; Ě.code=NIL}

Ě⟶Ě E , { Ě[0].place=endcons(Ě[1].place, E.place);
Ě[0].code=endcons(Ě[1].code, E.code) }



常量折叠与整数列表

- 如果一个表达式里元素对象全部是整数，那么编译时计算出表达式的值并替代它。
- I⟶i | I op I
- E⟶d | E op E | E op I | I op E | (E)

- 类似地考虑实数、以及推广到常量

- 整数列表（出现在数组声明语句中）
- Ǐ⟶i {Ǐ.val=list(getv(i))}
- Ǐ⟶Ǐ, i {
  Ǐ[0].val=endcons(Ǐ[1].val, getv(i))}

简单变量赋值语句翻译补：赋值表达式

- S⟶d=E　{
  S.code=E.code++gen[?getn(d)=?E.place]}

- 类型转换
- 设置type属性，d和E类型不一致时先转换后赋值
- S⟶d=E　{
  x=getn(d);
  type=lookup(x,type:);
  if(type==E.type)S.code=E.code++gen[?x= ?E.place];
  else if(type==INT)
  S.code=E.code++gen[?x ftoi ?E.place];
  else if(type==FLO)
  S.code=E.code++gen[?x itof ?E.place];
  else error()}

增量式代码生成生成四元式代码

- E⟶i {t=newvar(); E.place=t;
  emit(t '=' getv(i));}
- E⟶d {if(lookup(x)==UNBOUND)error();
  E.place=getn(d); }
- E⟶E op E {E[0].place=newvar();
  emit(E[0].place '=' E[1].place getn(op) E[2].place);}
- E⟶(E) {
  E[0].place=E[1].place;}
- E⟶-E {E[0].place=newvar();
  emit(E[0].place '= -' E[1].place]);}

使用了函数emit()，它的功能是根据实参生成一条三地址指令并添加到代码区尾部，该函数对实参个数不限，实参之间用空格分隔，要对实参求值。

10.3 布尔表达式简化为条件表达式C

- B⟶E | E r E | B ∧ B | B ∨ B | ! B | (B)
- 关系表达式是布尔表达式。
- 如果允许，算术表达式是布尔表达式
- 如果有布尔类型的话，布尔类型的常数、变量都是布尔表达式。
- 如果没有显式提供布尔类型，也可以像C语言那样将算术表达式作为布尔表达式。
- 布尔表达式通过布尔运算连接构成布尔表达式。

- 布尔类型常数只有真和假两个；
- 算术表达式被解释为布尔真（值为非0）和布尔假（值为0）
- 对布尔表达式求值采用短路算法。

布尔表达式的语义

- 关于运算符r及优先级

- C语言：
  - && || ! < == > <= >= !=
  - if(1 < i && i < 10) ...

- Pascal:
  - AND OR NOT < = > <= >=
  - if (x = 0) AND (a = 2) then...

- Fortran逻辑表达式:
  - .AND. .OR. .NOT. .LT. .EQ. .GT. .LE. .GE. .NE.
  - b = a .AND. 3 .LT. 5/2

布尔表达式的求值

- 短路算法：
  - A∨B　*if A then true else B*
  - A∧B　*if A then B else false*
  - ¬A　*if A then false else true*

```
if((p!=NULL)&&(p->val==0))…
```

- 作为条件控制用；
- 作为赋值语句的表达式用。

作为条件的布尔表达式

- 布尔表达式出现在分支、循环等控制语句中作为条件，
- 为此设计两类属性，名为tc和fc。
- tc和fc属性实例仅取值为标号，可以是多个标号，表示相应布尔表达式代码中为真转移去向，和为假时控制流流向。
- 对布尔表达式进行翻译将产生代码，用code属性来承载。

布尔表达式翻译

- B⟶E

  {l1=newl(); l2=newl();
  B.tc=list(l1);
  B.fc=list(l2);
  B.code=E.code++gen[IF ?E.place != 0 THEN ?l1 ELSE ?l2]}

- B⟶E r E

  {l1=newl(); l2=newl();
  B.tc=list(l1);
  B.fc=list(l2);
  B.code=E[1].code++E[2].code++
  gen[IF ?E[1].place ?getn(r) ?E[2].place THEN ?l1
  ELSE ?l2];}

布尔表达式翻译

- B⟶B ∧ B　{
  B[0].tc=B[2].tc;
  B[0].fc=append(B[1].fc, B[2].fc);
  B[0].code=B[1].code++
  　　genl[LABEL ?B[1].tc]++B[2].code}

- B⟶B ∨ B　{
  B[0].tc=append(B[1].tc, B[2].tc);
  B[0].fc=B[2].fc;
  B[0].code=B[1].code++
  　　genl[LABEL ?B[1].fc]++B[2].code}

- append(list1,list)将表list1和表list2合并为一个表，list1的表元素在前，list2的表元素在后，返回合并后的表。
- genl[LABEL ?llist]对llist中每个标号l都产生[LABEL ?l]

布尔表达式翻译

- B⟶! B {
  B[0].tc=B[1].fc;
  B[0].fc=B[1].tc;
  B[0].code=B[1].code}

- B⟶(B) {
  B[0].tc=B[1].tc;
  B[0].fc=B[1].fc;
  B[0].code=B[1].code}

举例

(123∨1<x)∧y⇨(E∨1<x)∧y E.place=t1;E.code=[t1=123]
⇨(B∨1<x)∧y B.tc=(l1);B.fc=(l2);B.code=[t1=123;IF t1!=0 THEN l1 ELSE l2]①
⇨(B∨E<E)∧y E[1].place=t2;E[1].code=[t2=1]; E[2].place=x;E[2].code=[]
⇨(B∨B)∧y B[2].tc=(l3);B[2].fc=(l4);B[2].code=[t2=1;IF t2<x THEN l3 ELSE l4]②
⇨(B)∧y B.tc=(l1,l3);B.fc=(l4);B.code=[①;LABEL l2;②]
⇨* B∧E E.place=y;E.code=[];
⇨B∧B B[2].tc=(l5);B[2].fc=(l6);B[2].code=[IF y!=0 THEN l5 ELSE l6]③
⇨B B.tc=(l5);B.fc=(l4 l6);B.code=[①;LABEL l2;②;LABEL l1; LABEL l3; ③]

注：句子(123∨1<x)∧y带注释的规范归约如上所示。每一行的注释部分为这步归约时进行求值的各属性及求值结果。其中变元的索引值表示对应句型中该变元的出现，从1开始算起。如第四行句型(B∨B)∧y的标注中使用B[2]指第二个B。对第一个B没有标注是因为在第二行标注过了，期间没有变化。其余类推。代码段后面带的序号，如第二行B.code的代码后附①，仅用于引用，如第5和8行引用该代码段来形成新的代码段。

例：对布尔表达式(123v1<x)∧y语义分析结果

- B.code=[
  t1=123;
  IF t1!=0 THEN l1 ELSE l2;
  LABEL l2;
  t2=1;
  IF t2<x THEN l3 ELSE l4;
  LABEL l1;
  LABEL l3;
  IF y!=0 THEN l5 ELSE l6]
- B.tc=(l5)
- B.fc=(l4 l6)

增量式代码生成

- B⟶E {B.tc=newl(); B.fc=newl(); emit[IF ?E.place !=0 THEN ?B.tc ELSE ?B.fc];}
- Ḃ⟶B∧ {emit[LABEL ?B.tc]; Ḃ .fc=B.fc;}
- Ḅ⟶B∨ {emit[LABEL ?B.fc]; Ḅ.tc=B.tc;}
- B⟶Ḃ B {B[0].tc=B[1].tc; B[0].fc=append(Ḃ.fc, B[1].fc); }
- E⟶Ḅ B {B[0].fc=B[1].fc; B[0].tc=append(Ḅ.tc, B[1].tc); }
- B⟶E r E {B.tc=newl(); B.fc=newl(); emit[IF ?E[1].place ?getn(r) ?E[2].place THEN ?B.tc ELSE ?B.fc];}
- B⟶(B) {B[0].tc=B[1].tc; B[0].fc=B[1].fc;}
- B⟶¬B {B[0].tc=B[1].fc; B[0].fc=B[1].tc;}

附注释

使用了宏emit[]，它的功能是根据实参生成一条三地址指令并添加到代码区尾部，该函数对实参个数不限，实参之间用空格分隔，不要对实参求值，除非参数带前边有？号。

对照函数emit()，它的功能是根据实参生成一条三地址指令并添加到代码区尾部，该函数对实参个数不限，实参之间用空格分隔，要对实参求值。

10.4 语句翻译

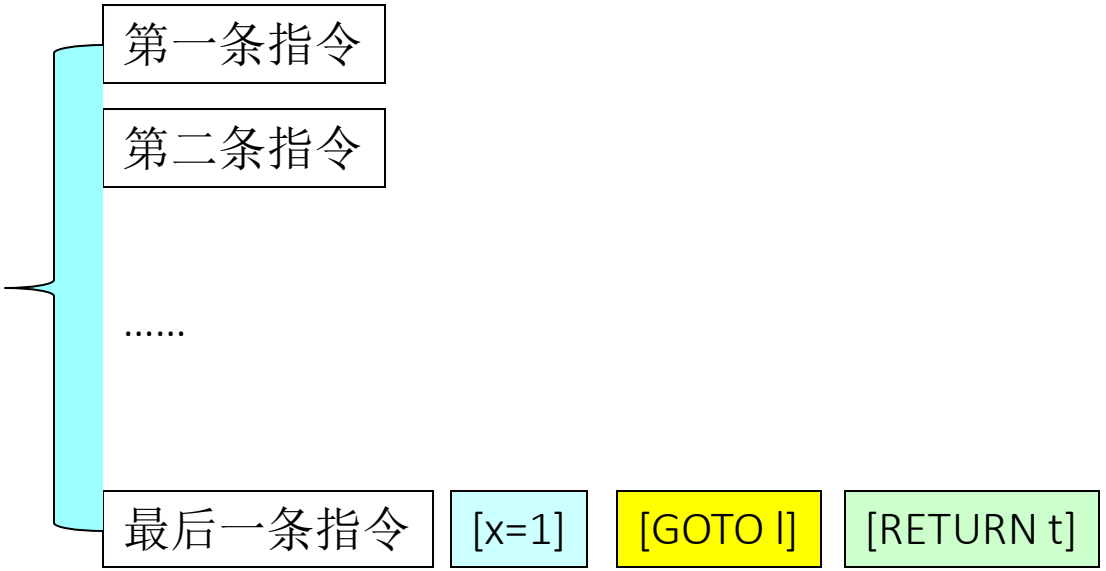
- 典型控制语句有顺序（复合语句）、分支和循环。
- 设计属性code承载语句翻译后产生的中间语言代码。
- 主要问题是对语句后继的访问如何进行处理。
- 采用综合属性替代继承属性完成语法树上的继承信息的处理。具体生成标号作为本语句在中间代码里所要访问的后继。这个标号将来会作为后继代码首地址（由祖先完成部署）。
- 概念：结构化代码块。只有一个入口（在最前）、只有一个出口（在最后）
- 由语句关心自己的后继，交给语境（上下文）去关心。

概念：结构化代码块

- 一个中间语言代码块是一个指令序列，存储为一片连续的存储区，序列中的相邻两条指令存放在相邻内存单元中，第一条指令对应于存储区第一个单元，最后一条指令对应存储区最后一个单元。这些构成指令的存储次序。
- 指令的控制流次序由代码块一次执行中的指令流来定义。指令流中第一条指令为代码块入口，指令流中最后一条指令为代码块出口
- 如果一个代码块的入口为第一条指令，出口为其最后一条指令，那么该代码块是结构化代码块
- S.code都是结构化代码块

语句的代码块都是结构化代码块

- D⟶Td(Ď){ĎŠ}
- S⟶for(S;B;S)S
- S⟶if(B)S else S
- S⟶if(B)S else S
- S⟶while(B)S
- S⟶{Š}
- S⟶goto lab
- S⟶d:S
- S⟶d=E
- S⟶d[Ě]=E
- S⟶d(Ř)
- S⟶return E



语句的代码、代码入口与后继

- S⟶if (B) S else S
  - S[0].next=append(S[1].next, S[2].next)； S[1].entry=B.tc；S[2].entry=B.fc
  - S[0].code=B.code++genl[LABEL, ?B.tc]++S[1].code++gen[GOTO ?S[1].next]++genl[LABEL, ?B.fc]++S[2].code

- S⟶while (B) S
  - S[1].entry=B.tc； S[0].next=B.fc； S[1].next=B.entry
  - S[0].code=gen[LABEL ?B.entry]++B.code++genl[LABEL, ?B.tc]++S[1].code++gen[GOTO ?B.entry]++genl[LABEL ?B.fc]

语句的代码、代码入口与后继

- S⟶{Š}
  - S.next=Š.next； S.entry=Š.entry
  - S.code=merg_code(Š.code)
- Š⟶S
  - Š.next= S.next； Š.entry=S.entry
  - Š.code=list(S.code)
- Š⟶ŠS;
  - Š[0].next=S.next； Š[0].entry=Š[1].entry；
    Š[1].next=S.entry
  - Š[0].code= endcons(S.code, Š[1].code)

赋值语句翻译

- S⟶d = E { //前面已介绍过
  x=getn(d);
  if(lookup(x)==UNBOUND)error();
  S.code=E.code++gen[?x= ?E.place]}

- S⟶d[Ě] = E {?}//见数组声明与下标变量处理节

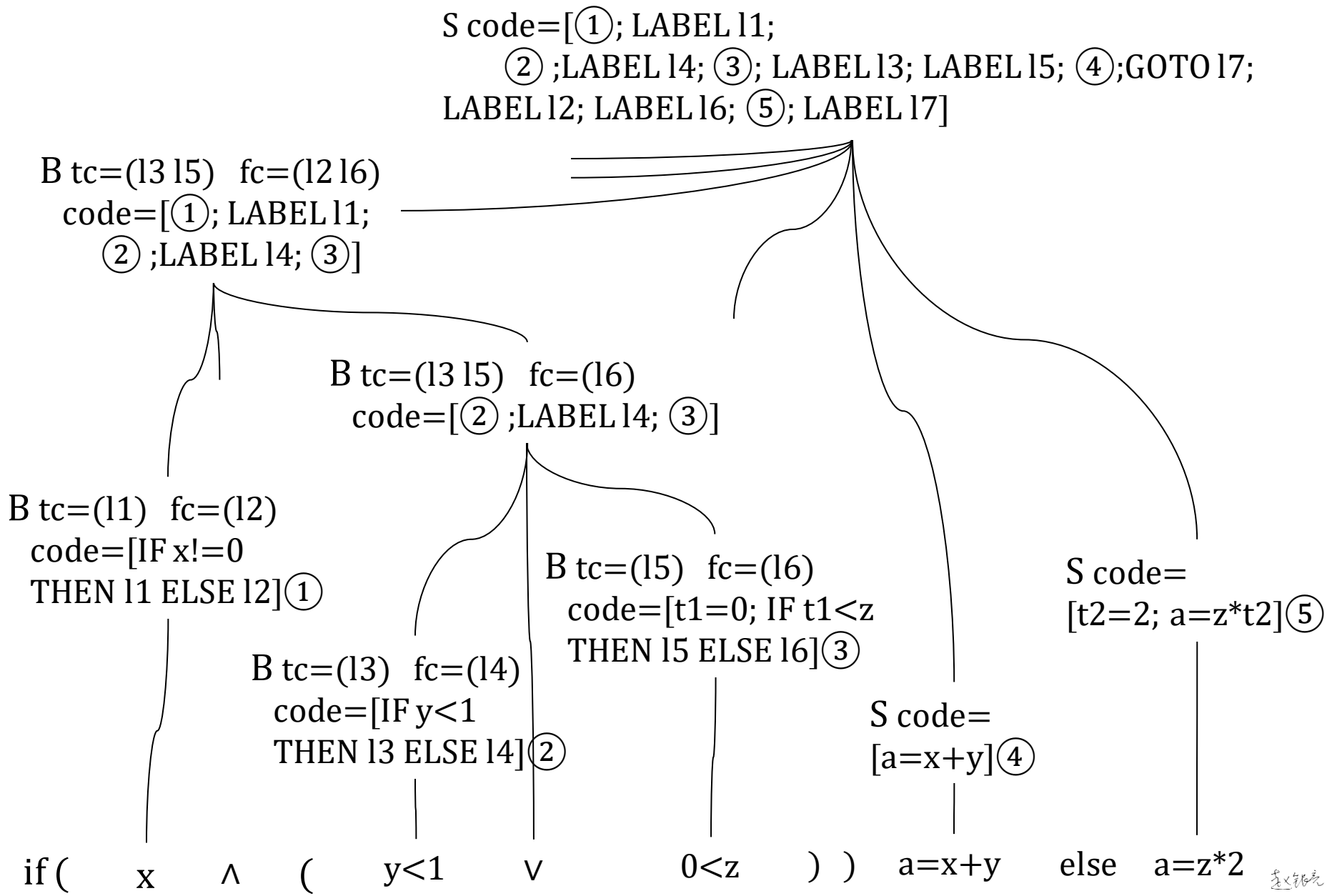
分支语句翻译

- S⟶ if (B) S {
  S[0].code=B.code++
  genl[LABEL ?B.tc]++
  S[1].code++
  genl[LABEL ?B.fc]}

if(x<y)x=y
⇨ if(B)x=y B.tc=(l1) B.fc=(l2) B.code =[IF x<y THEN l1 ELSE l2]
⇨if(B)S S.code=[x=y]
⇨S S.code=[IF x<y THEN l1 ELSE l2; LABEL l1; x=y; LABEL l2]

- S⟶if (B) S else S {
  l=newl();
  S[0].code=B.code++
  genl[LABEL ?B.tc]++
  S[1].code++
  gen[GOTO ?l]++genl[LABEL ?B.fc]++
  S[2].code++
  gen[LABEL ?l]}

举例：布尔表达式作为if语句的条件



循环语句翻译

- S⟶ while (B) S {
  l=newl();
  S[0].code=
  gen[LABEL ?l]++
  B.code++
  genl[LABEL ?B.tc]++
  S[1].code++
  gen[GOTO ?l]++
  genl[LABEL ?B.fc]}　　//结构化块

例

- if (x<y) while (z) {x=1; y=b}
- x<y⇨B
  B.tc=l1;B.fc=l2;B.code=[IF x<y THEN l1 ELSE l2]
- x=1⇨S　　S.code=[t1=1;x=t1]
- y=b⇨S　　S.code=[y=b]
- {S;S}⇨ S　　S.code=[t1=1;x=t1;y=b]②
- z⇨B
- B.tc=l3;B.fc=l4;B.code=[IF z!=0 THEN l3 ELSE l4]①
- while (B) S⇨S
- S.code=[LABEL l5;①;LABEL l3;②;GOTO l5;LABEL l4]③
- if(B)S⇨S
- S.code=[IF x<y THEN l1 ELSE l2; LABEL l1;③;LABEL l2]

例子

- if (x<y) while (z) {x=1; y=b}⇨*S
- IF x<y THEN l1 ELSE l2; LABEL l1;③;LABEL l2，其中③为，LABEL l5;①;LABEL l3;②;GOTO l5;LABEL l4，所以，
- S.code=[
  IF x<y THEN l1 ELSE l2;
  LABEL l1;
  LABEL l5;
  IF z!=0 THEN l3 ELSE l4;
  LABEL l3;
  t1=1;
  x=t1;
  y=b;
  GOTO l5;
  LABEL l4;
  LABEL l2]

复合语句翻译

- S⟶{Š} {
  S.code=merg_code(Š.code);}

- Š⟶S {Š.code=list(S.code);}

- Š⟶Š;S {Š.code=endcons(S.code, Š[1].code);}

增量式代码生成

- C⟶ if (B) { emit[LABEL ?B.tc]; C.next:= E.fc; }
- T^p⟶ C S else { T^p.next=newl();
  emit[GOTO ? T^p.next];emit[LABEL C.next];}
- S⟶ T^p S { emit[LABEL ?T^p.next];}
- W⟶while { W.addr=newl(); emit[LABEL W.addr];}
- W^d⟶W (B) {emit[LABEL ?B.next]; W^d.addr=W.addr;
  W^d.next=B.fc;}
- S⟶W^d S {emit[GOTO ?W^d.addr];emit[LABEL ?W^d.next];}

注：emit[LABEL ?ll]的参数ll为一个表，该表的每个元素是一个标号，那么对于ll中每个标号，依次连续生成LABEL指令。比如ll=‘(l1, l2, l3)，那么生成3条LABEL指令，即为[LABEL l1; LABEL l2; LABEL l3] 。

高级语言的goto语句与带标号语句

- S→goto d
- S→d:S
- 标号先定义后引用
- lab1:if(x<1)…
- …
- goto l1

- 标号先引用后定义
- goto l2
- …
- goto l2
- …
- lab2:…

- S→d:S
  - d标号不存在：
    建立登记项并生成标号l1置于place域；
    生成[LABEL l1]++S[1].code
  - d标号存在；
    - 定义否？未定义：
      值定义否为已定义；取place域值l2
      生成[LABEL l2]++S[1].code
    - 定义否？已定义：标号重定义错
- S→goto d
  - d标号不存在：
    建立登记项并生成标号l1置于place域；
    生成[GOTO l1]
  - d标号存在（定义否？未/已定义）：
    取place域值l2生成[GOTO l2]

goto语句翻译

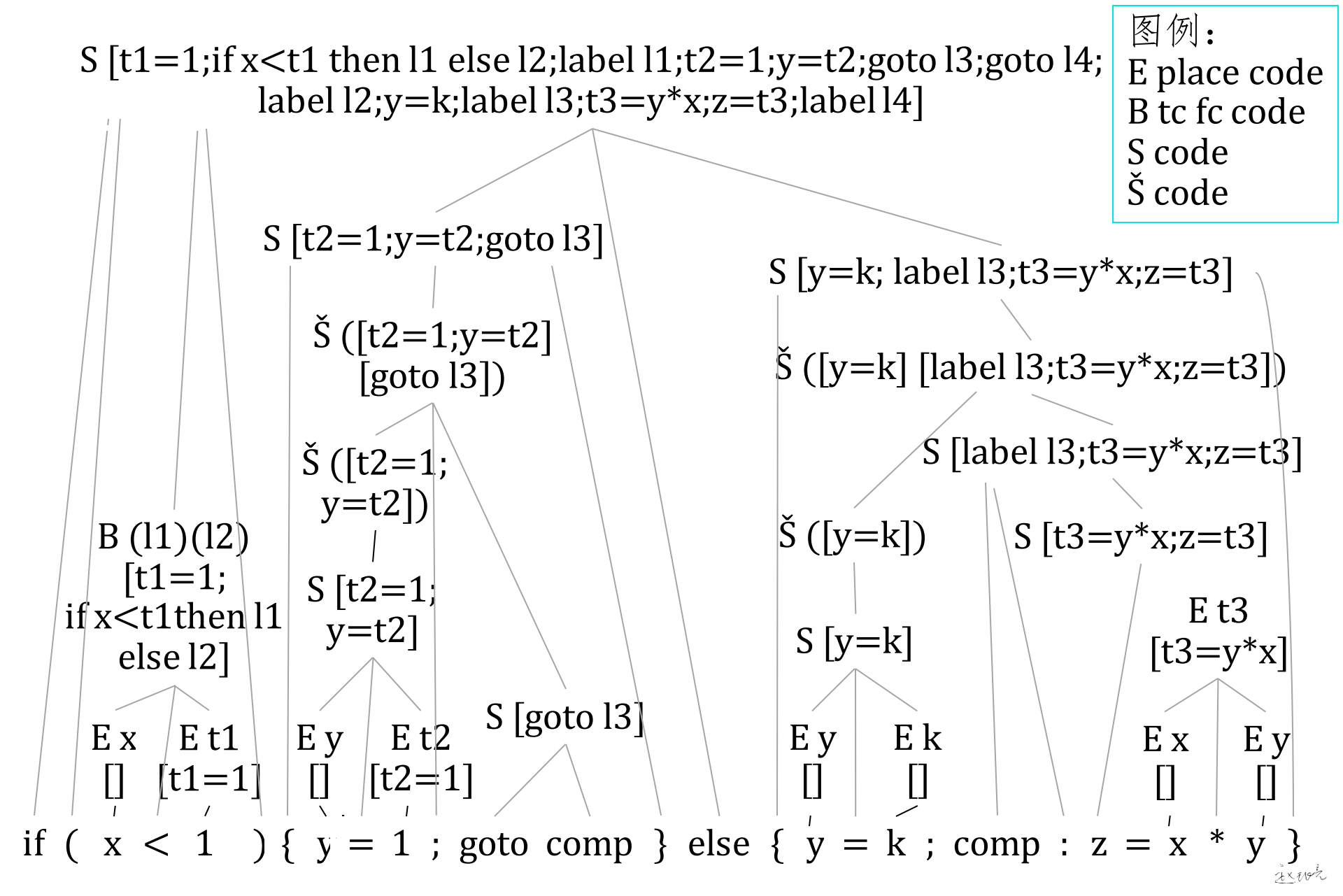
- S→goto d {

```
l1=newl();
x=getn(d);
if(lookup(x)==UNBOUND){//标号的引用先于定义
        bind(x, LAB);
        lookup(x, place:l1);
        lookup(x, def:NO);
        gen[GOTO ?l1]}
else{//标号已经不是第一次被引用
        l2=lookup(x, place);//标号肯定存在
        S.code=gen[GOTO ?l2]}}
```

带标号语句翻译

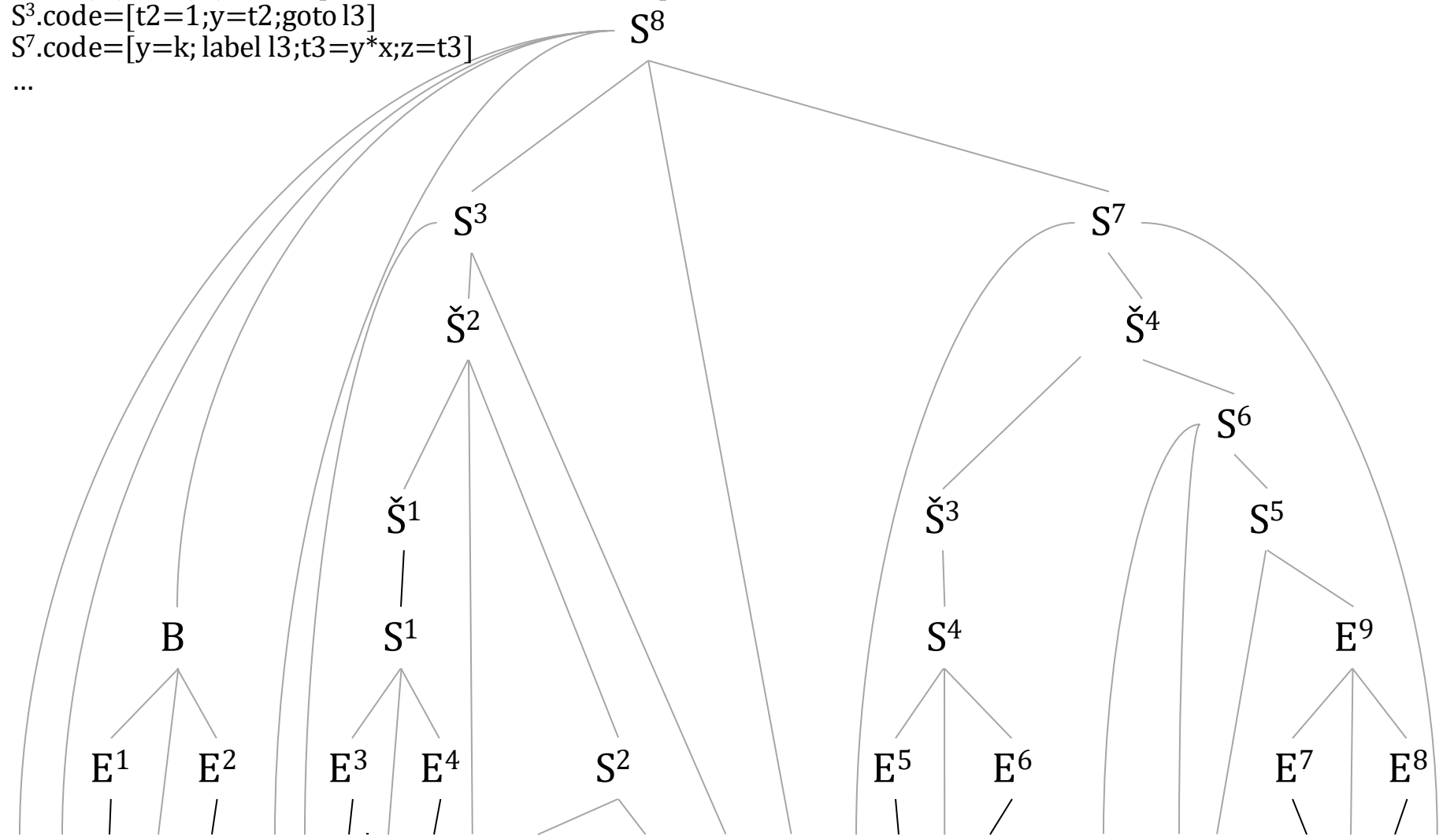
- S⟶d:S　　{
  l1=newl();
  x=getn(d);
  if(lookup(x)==UNBOUND){//标号定义在先
  bind(x,LAB);
  lookup(x,place:l1);
  lookup(x,def:YES);
  S[0].code=gen[LABEL ?l1]++S[1].code}
  else if(lookup(x,type)==LAB&&lookup(x,def)!=YES){
  l2=lookup(x,place);
  lookup(x,def:YES);
  S[0].code=gen[LABEL ?l2]++S[1].code}
  else error()}

例：语句翻译


图例：
E place code
B tc fc code
S code
Š code
S [t1=1;if x<t1 then l1 else l2;label l1;t2=1;y=t2;goto l3;goto l4;
label l2;y=k;label l3;t3=y*x;z=t3;label l4]
S [t2=1;y=t2;goto l3]
S [y=k; label l3;t3=y*x;z=t3]
Š ([t2=1;y=t2]
[goto l3])
Š ([y=k] [label l3;t3=y*x;z=t3])
Š ([t2=1;
y=t2])
S [label l3;t3=y*x;z=t3]
B (l1)(l2)
[t1=1;
if x<t1then l1
else l2]
S [t2=1;
y=t2]
Š ([y=k])
S [t3=y*x;z=t3]
S [y=k]
E t3
[t3=y*x]
S [goto l3]
E x
[]
E t1
[t1=1]
E y
[]
E t2
[t2=1]
E y
[]
E k
[]
E x
[]
E y
[]
if (x < 1) { y = 1 ; goto comp } else { y = k ; comp : z = x * y }

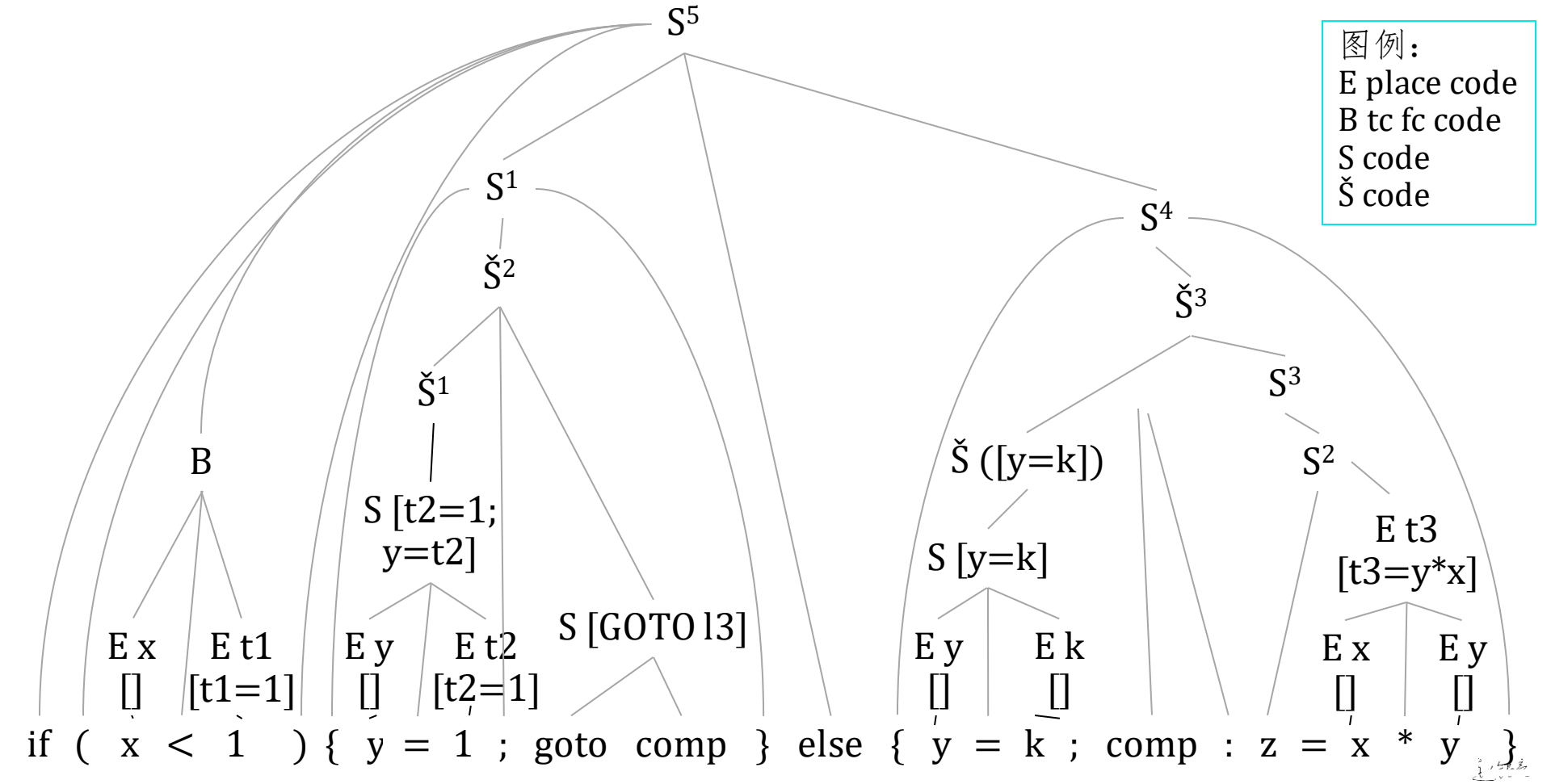
注释语法树

S^8 [t1=1;if x<t1 then l1 else l2;label l1;t2=1;y=t2;goto l3;goto l4;label l2;y=k;label l3;t3=y*x;z=t3;label l4]
B.tc=(l1) fc=(l2) code=[t1=1;if x<t1then l1 else l2]
S^3.code=[t2=1;y=t2;goto l3]
S^7.code=[y=k; label l3;t3=y*x;z=t3]
...



注释语法树

S^5 [t1=1;IF x<t1 THEN l1 ELSE l2;LABEL l1;t2=1;y=t2;GOTO l3;GOTO l4;LABEL l2;y=k;LABEL l3;t3=y*x;z=t3;LABEL l4]　　B.tc=(l1) fc=(l2) code=[t1=1;IF x<t1THEN l1 ELSE l2]
S^1.code=[t2=1;y=t2;GOTO l3]　S^2.code=[t3=y*x;z=t3]　S^3.code=[LABEL l3;t3=y*x;z=t3]
S^4.code=[y=k; LABEL l3;t3=y*x;z=t3]　　$Š^1$.code= ([t2=1;y=t2])
$Š^2$.code=([t2=1;y=t2] [GOTO l3])　　$Š^3$.code= ([y=k] [LABEL l3;t3=y*x;z=t3)



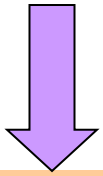
10.5 下标变量引用

- 例：int a[10,5,20]
- f@table: (width: 4000 … entry: (name: a type: ARRAY base: 4000 etype: INT dims: 3 dim[0]: 10 dim[1]: 5 dim[2]: 20)…)

- 为左值引用S⟶a[Ě]=E生成代码
- 为右值引用E⟶a[Ě]生成代码

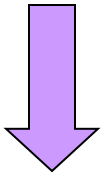
下标变量的语义

a[i+1]



a + (i+1) * sizeof(int)

a[*t*]



base_address(a) + (*t* - lower_bound(a)) * element_size(a)

(1) 地址计算公式

- 对于一维数组a[i]：
  - 下标的变化范围：$l \leq i < u$
  - 连续存储区的首址：base，每个数组元素占用w个单元
  - 则a[i]地址为：base+(i-l)*w

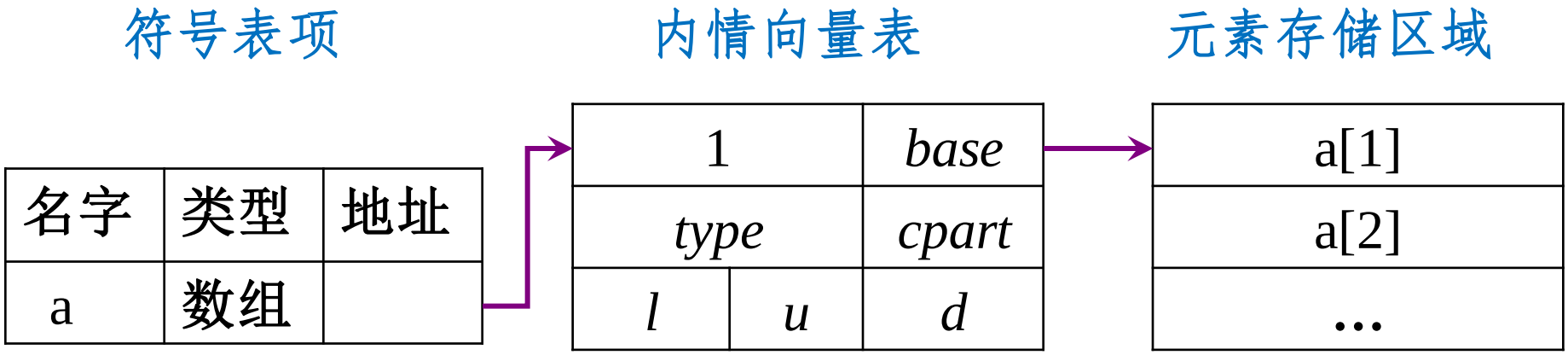
- a[i]地址：base+(i-l)*w=(base-l*w)+i*w
- 固定部分：base-l*w　　//编译时确定
- 变化部分：i*w　　//运行时确定

- 如果l恒为0，那么a[i]地址：base+i*w

(2) 数组声明a[l:u]的信息

- 符号表登记项：
  - name、type=ARRAY、etype、base、dims、dim[i]
  - 允许下界不为0则需要更多类型特有域：cpart、lb[i]、ub[i]

---书的采用---



- 对于一维数组元素引用形式a[i]，该元素的存储位置为：

$$base+(i-l)*w = base - cpart + i*w$$

地址计算公式（二维）

- 对于二维数组a[i,j]：
  - 下标的变化范围：$l_1 \le i < u_1$; $l_2 \le j < u_2$
  - 连续存储区的首址：$base$，每个数组元素占用w个单元
  - 则a[i,j]地址为：$base+[(i-l_1)*(u_2-l_2)+j-l_2]*w$

$$base+[(i-l_1)*(u_2-l_2)+j-l_2]*w$$

$$=[base+(-l_1*(u_2-l_2)-l_2)*w] + [(i*(u_2-l_2)+j)*w]$$

固定部分：$base$；下界恒为0

变化部分：$(i*(u_2-l_2)+j)*w$

用维长表示：$(i*d_2+j)*w$

地址计算公式（一般情形）

- 对于多维数组a[i_1,i_2,...,i_n]：
  - 下标的变化范围：$l_1 \le i_1 < u_1$;...; $l_n \le i_n < u_n$
  - 连续存储区的首址：*base*，每个数组元素占用单元*w*个
- 则a[i_1,i_2,...,i_n]地址为：
- $base+((...((i_1-l_1)*d_2+(i_2-l_2))*d_3+(i_3-l_3))...)d_n+(i_n-l_n))*w$

$$=base-((...((l_1*d_2+l_2)*d_3+l_3)...)d_n+l_n)*w+$$

$$i_1*d_2*...*d_n+i_2*d_3*...*d_n+...+i_n$$

$$= base - cpart +$$

$$(...((i_1)*d_2+i_2)*d_3+i_3)*d_4+i_4)...)*d_n+i_n$$

- 如果下界恒为0，则*cpart*为0，只需为运行时产生代码对应于可变部分。这是本课程采用的情形。

数组元素引用

- E⟶d[Ě]
- a[i+1, j*2, 4]
- Ě.code是一个表，表元素同E.code，顺序与列表一致。
  - ([t1=i+1] [t2=j*2] [t3=4])
- Ě.place是一个表，表元素同E.place，顺序与列表一致。
  - (t1 t2 t3)　　//为各表达式的结果变量
- E.code用于存放由表达式翻译产生的中间代码。
  - 计算地址：$(t1*d_2+t2)*d_3+t3$
  - [t1=i+1;t2=j*2; t3=4;t100=t1*5;t101=t100+t2; t102=t101*20;t103=t102+t3; t104=t103*4;t105=a+t104;t106=M[t105]]
- E.place中存放结果变量t106，是上述计算下标变量地址并引用的代码的执行结果。

数组元素引用（右值）

```
▻ E⟶d[Ě]      {
1  ilist=Ě.place;
2  t0=car(ilist); ilist=cdr(ilist);
3  code=[]; x=getn(d);
4  dims=lookup(x, dims:);
5  for(i=1;i<dims;i++){
6          t=newvar();
7          dim=lookup(x, dim[i]:);
8          code++=gen[?t=?t0 *?dim)];
9          t0=newvar();
10         code++=gen[?t0=?t+?car(ilist)];
11         ilist=cdr(ilist);
12         }
13 t=newvar();
14 code++=gen[?t=?t0*?sizeof(lookup(x,etype:))]
14 t0=newvar(); t=newvar(); E.place=t;
15 E.code=merg_code(Ě)++code++gen[?t0=?x+?t;  ?t=M[?t0]]}
```

设Ě.place= ‘(t1 t2 t3)
t0=‘t1; code=[]; x=‘a; dims=3
t=‘t100; dim=5
code++=[t100=t1*5]
t0=‘t101
code+=[t101=t100+t2]
t=‘t102; dim=20
code++=[t102=t101*20]
t0=‘t103
code++=[t103=t102+t3]
t=‘t104
code++=[t104=t103*4]
t0=‘t105; E.place=‘t105
E.code=[t1…;t2…;t3…;t100…;t1
01…;t102…;t103…;t104…;
t105=a+t104; t106=M[t105]]

数组元素引用（左值）

```
▻ S⟶d[Ě]=E                {
1  ilist=Ě.place;
2  t0=car(ilist); ilist=cdr(ilist);
3  code=[]; x=getn(d);
4  dims=lookup(x, dims:);
5  for(i=1;i<dims;i++){
6          t=newvar();
7          dim=lookup(x, dim[i]:);
8          code++=gen[?t=?t0 *?dim];
9          t0=newvar();
10         code++=gen[?t0=?t+?car(ilist)];
11         ilist=cdr(ilist);
12         }
13  t=newvar(); t1=newvar();
14 code++=gen[?t=?t0*?sizeof(lookup(x,etype:))]
15 S.code=merg_code(Ě.code)++E.code++code++
16         gen[?t1=?x+?t; M[?t1]=?E.place]}
```

```
Ě.place='(t5 t6 t7) E.place='t8
t0='t5; code=[]; x='a; dims=3
t='t100; dim=5
code++=[t100=t5*5]
t0='t101
code+=[t101=t100+t6]
t='t102; dim=20
code++=[t102=t101*20]
t0='t103
code++=[t103=t102+t7]
t='t104
code++=[t104=t103*4]
S.code=[t5...;t6...;t7...]++
E.code++[t100...;t101...;t102...;
t103...;t104...; t105=a+t104; M
[t105]=t8]
```

数组元素引用

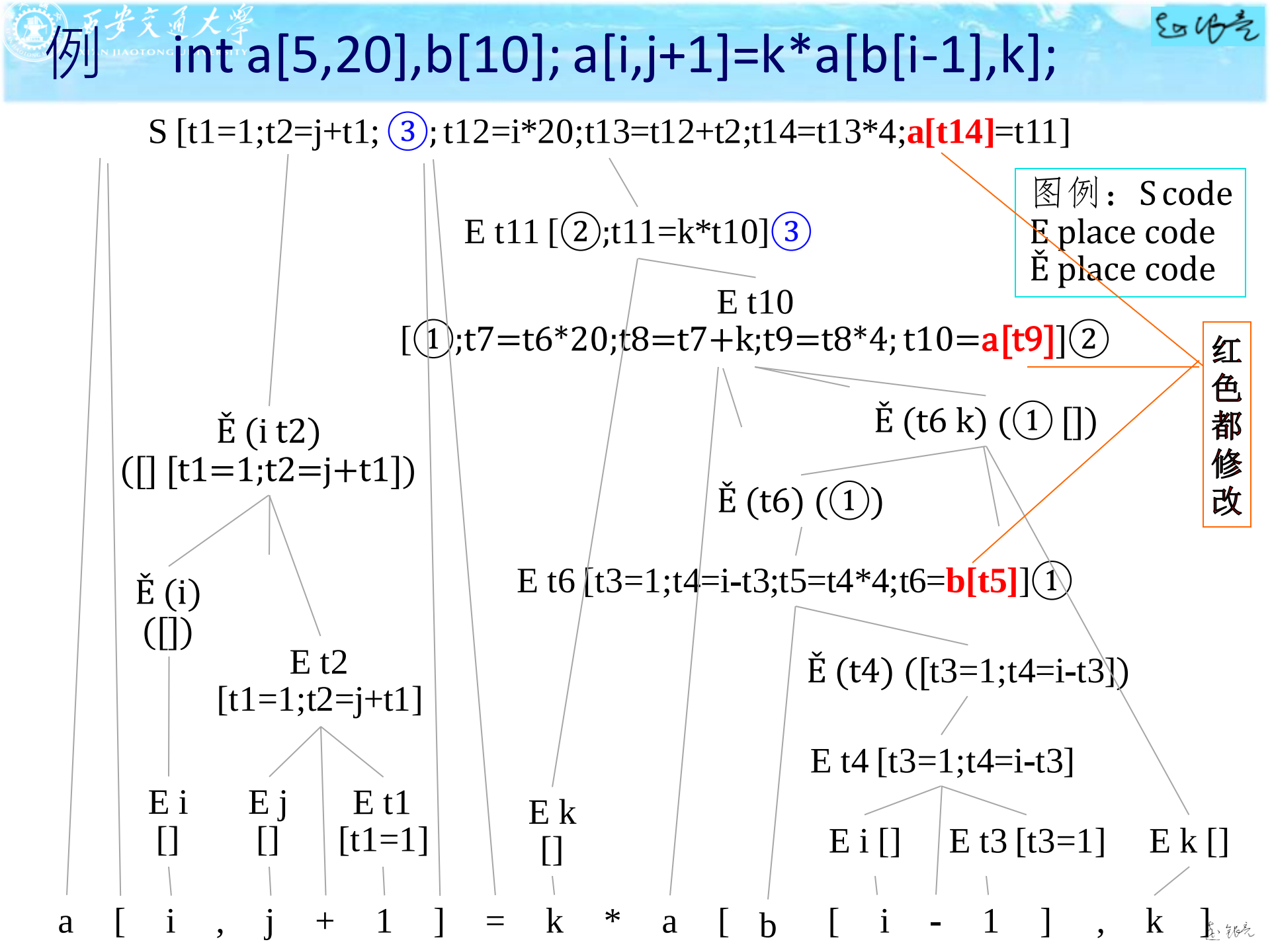
- S⟶d[Ě]=E
- a[i+1, j*2, 4]=66
- Ě.code是一个表，表元素同E.code，顺序与列表一致。
  - ([t5=i+1] [t6=j*2] [t7=4])
- Ě.place是一个表，表元素同E.place，顺序与列表一致。
  - (t5 t6 t7)
- E.code用于存放由表达式翻译产生的中间代码。
- E.place中存放一个变量t8，这个变量存放代码的执行结果。
- S.code语句S的代码
  - 计算地址：$(t5*d_2+t6)*d_3+t7$
  - [t5=i+1;t6=j+2;t7=4;t8=66;t100=t5*5;t101=t100+t6;t102=t101*20;t103=t102+t7;t104=t103*4;t105=a+t104;M[t105]=t8]

带注释的规范归约过程

- a[i+1, j*2, 4,]=66
- ⇦ a[Ě i+1, j*2, 4,]=66 Ě.place=NIL Ě.code=NIL
- ⇦ a[Ě E, j*2, 4,]=66 E.place='t5 E.code=[t5=i+1]
- ⇦ a[Ě j*2, 4,]=66 Ě.place='(t5) Ě.code='([t5=i+1])
- ⇦ a[Ě E, 4,]=66 E.place='t6 E.code=[t6=j*2]
- ⇦ a[Ě 4,]=66 Ě.place='(t5 t6) Ě.code='([t5=i+1] [t6=j*2])
- ⇦ a[ĚE,]=66 E.place='t7 E.code=[t7=4]
- ⇦ a[Ě]=66 Ě.place='(t5 t6 t7) Ě.code='([t5=i+1] [t6=j*2] [t7=4])
- ⇦ a[Ě]=E E.place='t8 E.code=[t8=66]
- ⇦ S

  S.code=[t5=i+1;t6=j*2;t7=4;t8=66;t100=t5*5;t101=t100+t6;t102=t101*20; t103=t102+t7;t104=t103*4; t105=a+t104; M[t105]=t8]

例 int a[5,20],b[10]; a[i,j+1]=k*a[b[i-1],k];



int a[5,20],b[10]; a[i,j+1]=k*a[b[i-1],k];

[t1=1
t2=j+t1
t3=1
t4=i-t3
t5=t4*4
t6=**b[t5]** ✗
t7=t6*20
t8=t7+k
t9=t8*4
t10=**a[t9]** ✗
t11=k*t10
t12=i*20
t13=t12+t2
t14=t13*4
a[t14]=t11] ✗

[t1=1
t2=j+t1
t3=1
t4=i-t3
t5=t4*4
t6=b+t5
t7=M[t6]
t8=t7*20
t9=t8+k
t10=t9*4
t11=a+t9
t12=M[t11]
t13=k*t12
t14=i*20
t15=t14+t2
t16=t15*4
t17=a+t16
M[t17]=t13]

翻译为中间表示：QTAC代码

{int z; int a[10,20]; int bar(int x;){return x+1;}
float foo(int x; int b[]; int boo();){if(x>0)z=b[0]+b[10]; else return boo(b[20],)};
print foo(0, a[], &bar,);}

```
i4 z;
i800 a;
define bar(x){LABEL bar; t1=x+1; return t1;}
define foo(x, b, boo){
        LABEL foo;
        IF x>0 THEN l1 ELSE l2; LABEL l1;
        t2=0*4; t3=b+t2; t4=M[t3];
        t5=10*4; t6=b+t5; t7=M[t5];
        z=t4+t7; GOTO l3;
        LABEL l2; t8=20*4; t9=b+t8; t10=M[t9];
        PAR t10; t11=CALL boo, 1; RETURN t11;
        LABEL l3;}
PAR 0; PAR a; PAR bar; t12=CALL foo, 3; PRINT t12;
```

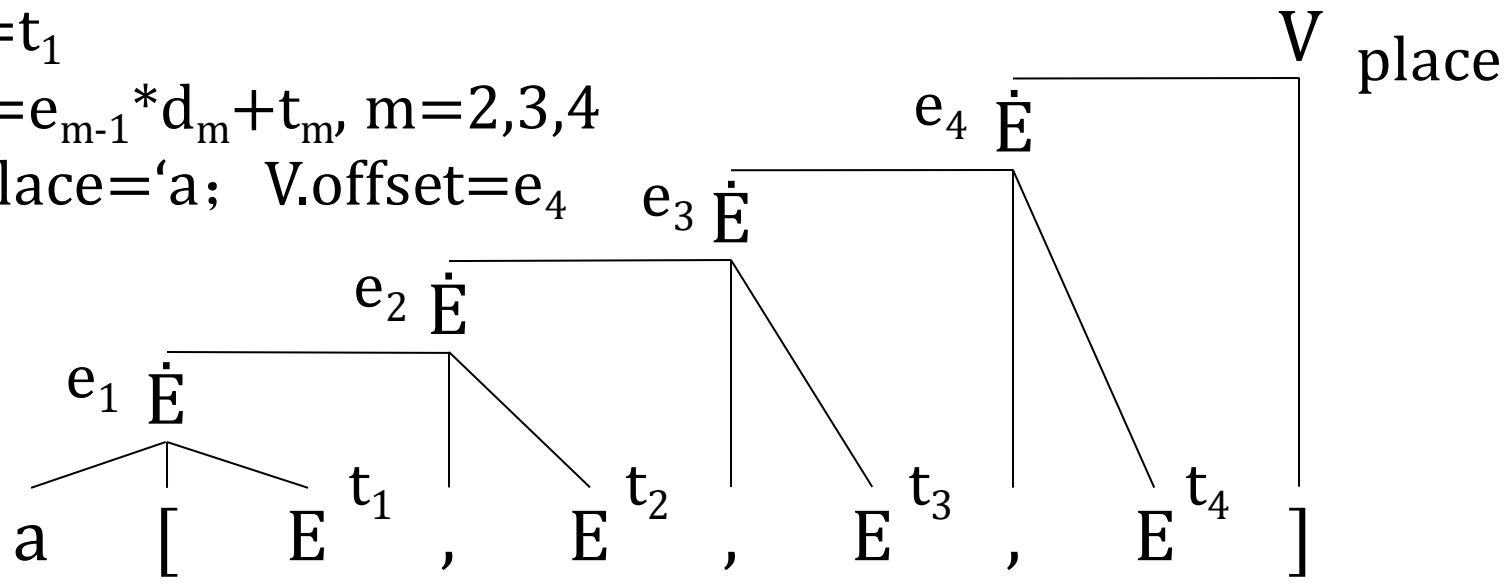
西安交通大学赵银亮
《形式语言与编译》

对下标变量的处理增量式代码生成

V⟶Ė] | d
Ė⟶Ė,E | d[E

- $e_1=t_1$
- $e_m=e_{m-1}*d_m+t_m$

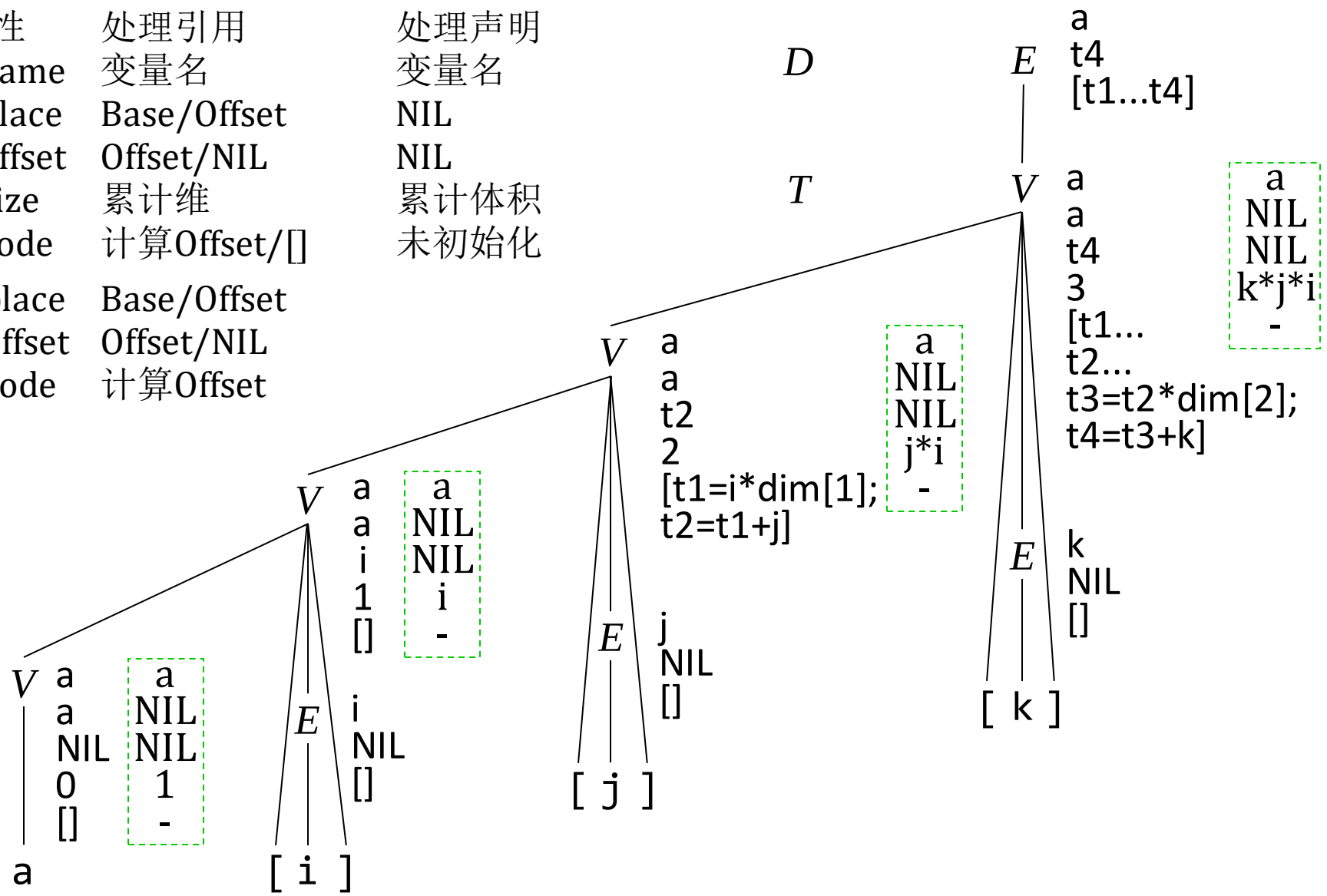
$e_1=t_1$
$e_m=e_{m-1}*d_m+t_m$, m=2,3,4
V.place=‘a； V.offset=e_4



西安交通大学赵银亮
《编译语言与编译》

对数组变量引用与声明的语义分析

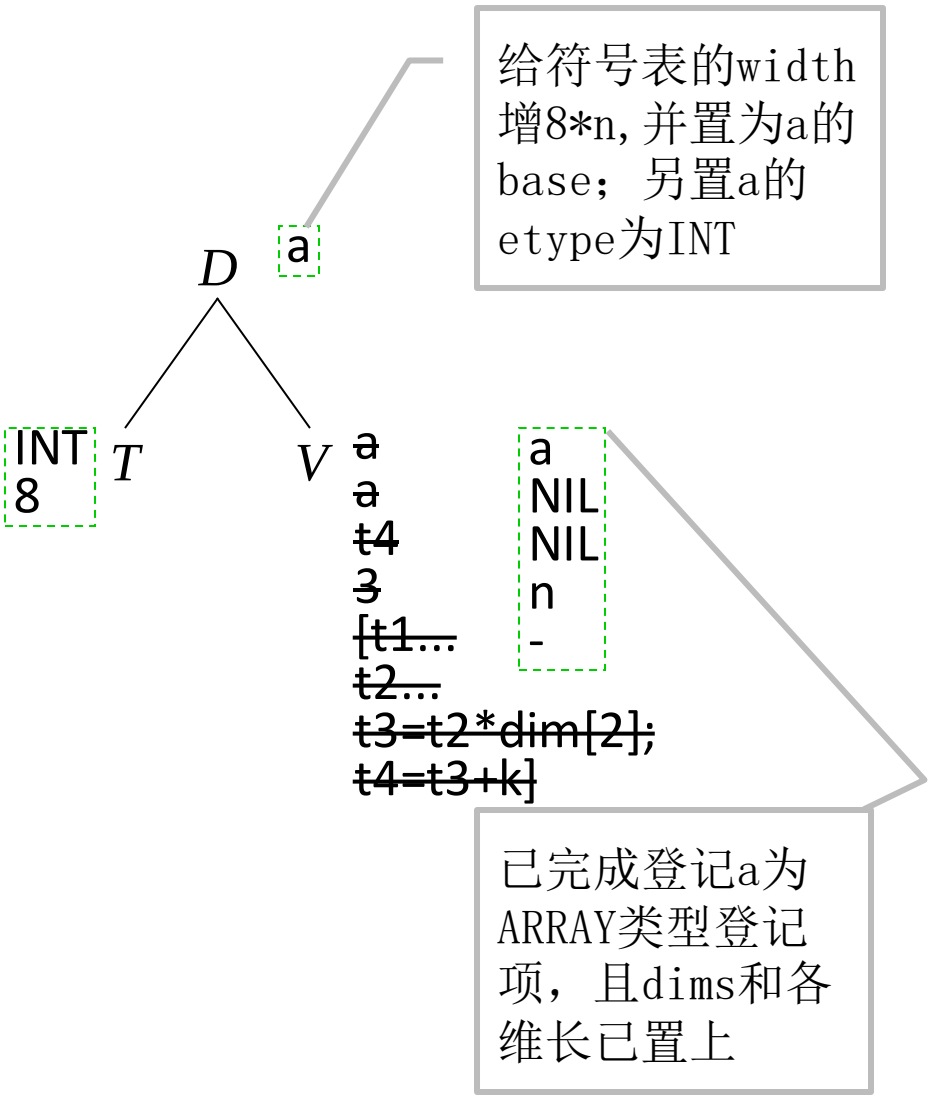
属性	处理引用	处理声明
V.name	变量名	变量名
V.place	Base/Offset	NIL
V.offset	Offset/NIL	NIL
V.size	累计维	累计体积
V.code	计算Offset/[]	未初始化
E.place	Base/Offset	
E.offset	Offset/NIL	
E.code	计算Offset	



西安交通大学赵银亮
《形式语言与编译》

对数组变量声明的语义分析

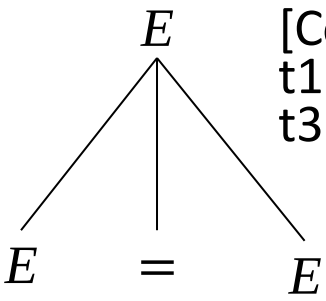
属性	处理引用	处理声明
V.name	变量名	变量名
V.place	Base/Offset	NIL
V.offset	Offset/NIL	NIL
V.size	累计维	累计体积
V.code	计算Offset/[]	未初始化
E.place	Base/Offset	
E.offset	Offset/NIL	
E.code	计算Offset	
T.name		类型名
T.size		类型尺寸
D.place		标识符



西安交通大学赵银亮
《形式语言与编译》

对变量引用的语义分析

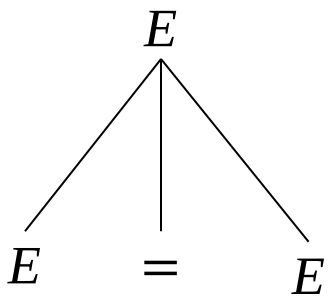
属性	处理引用	处理声明
V.name	变量名	变量名
V.place	Base/Offset	NIL
V.offset	Offset/NIL	NIL
V.size	累计维	累计体积
V.code	计算Offset/[]	未初始化
E.place	Base/Offset	
E.offset	Offset/NIL	
E.code	计算Offset	



a
t_a
[Code_a]

b
t_b
[Code_b]

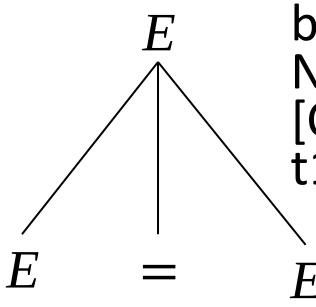
t2
NIL
[Code_a; Code_b;
t1=b+t_b; t2=M[t1];
t3=a+t_a; M[t3]=t2]



a
NIL
[Code_a]

b
t_b
[Code_b]

a
NIL
[Code_a; Code_b;
t1=b+t_b; t2=M[t1];
a=t2]



a
t_a
[Code_a]

b
NIL
[Code_b]

b
NIL
[Code_a; Code_b;
t1=a+t_a; M[t1]=b]

10.6 函数调用语句

- 函数声明$D \longrightarrow Td(\check{A})\{\check{D}\check{S}\}$
  - $A \longrightarrow E$
  - $A \longrightarrow d[] \mid d[\check{I}]$
  - $A \longrightarrow d() \mid d(\check{T})$
- 函数调用 $S \longrightarrow d(\check{R})$ 和 $E \longrightarrow d(\check{R})$
  - 调用形式与返回类型有关
  - $R \longrightarrow E$
  - $R \longrightarrow d[]$
  - $R \longrightarrow \&d$ 或 $R \longrightarrow d()$
- QL文法$E \longrightarrow d(\check{R})$

- “先声明后引用原则”

先声明后引用原则

- 什么是“声明”？
  - 告诉编译器：这个名字（变量 / 函数 / 类）存在，它是什么类型。
  - （不分配内存 / 不实现具体逻辑）
- 什么是“引用”？
  - 使用这个名字：调用、赋值、访问。
- 编译器必须先看到声明，才能允许你使用（引用）。
  - 否则编译器会报：未定义标识符 / 找不到符号。

访问符号表登记项

- lookup()类似只是默认symtab栈顶符号表
- lookup1(tab,name)查找name返回该登记项或UNBOUND
- lookup1(tab,name, nature:)返回name登记项的nature域值
- lookup1(tab,name, nature:, value)置name登记项的nature域值为value
- lookup1(tab,name, nature[index]:, value) 置name登记项的nature域的第index个元素的值为value

- bind()同bind1()只是默认symtab栈顶符号表
- bind1(tab,name, type)将name加入tab作为一个登记项类型为type，若已存在则返回错误

- 抽象为统一形式：根据第一个实参是否为符号表指针进行区分，若是表示对指定符号表的操作，否则为默认符号表。

作为右值的函数调用翻译

- $E \longrightarrow d\ (\check{R})$ {

```
code=[];
parlist= Ř.place;
x=getn(d);
argc=length(parlist); //实参个数
while(parlist!=NIL){
      code=gen[PAR ?car(parlist)]++code;   //采用实参倒序
      parlist=cdr(parlist);}
v=newvar();
code++=gen[?v=CALL ?x, ?argc];
E.place=v;//该变量存放返回结果
E.code=merg_code(Ř.code)++code;
}
```

函数调用语句

- $S \longrightarrow d\ (\check{R})$ {

```
code=[];
parlist= Ř.place;
x=getn(d);
argc=length(parlist);
while(parlist!=NIL){
        code=gen[PAR ?car(parlist)]++code;
        parlist=cdr(parlist);}
v=newvar();
code++=gen[?v=CALL ?x, ?argc];
S.code=merg_code(Ř.code)++code;
}
```

数组原型与函数原型作实参

- $\check{R} \longrightarrow \varepsilon \{\check{R}.\text{place}=\text{NIL}; \check{R}.\text{code}=\text{NIL}\}$
- $\check{R} \longrightarrow \check{R}R, \{\check{R}[0].\text{place}=\text{endcons}(R.\text{place}, \check{R}[1].\text{place});$
  $\check{R}[0].\text{code}=\text{endcons}(R.\text{code}, \check{R}[1].\text{code})\}$
- $R \longrightarrow E \ \{R.\text{place}=E.\text{place}; R.\text{code}=E.\text{code}\}$
- $R \longrightarrow d[] \{R.\text{place}=\text{getn}(d); R.\text{code}=[]\}$//可加检查形参
- $R \longrightarrow \&d \{R.\text{place}=\text{getn}(d); R.\text{code}=[]\}$//可加检查形参
- $E \longrightarrow d(\check{R})$//由此看到如何区分无参函数调用与函数原型
- $S \longrightarrow d(\check{R})$

考虑到数组和函数作为实参，形式为$d[]$和$\&d$，不能归约为E，因为既不是数组元素引用也不是函数调用。所以在主文法中归约为R。相应地采用这些文法来描述实参。

为了简单期间，对于实参$d[]$和$\&d$，只要写为数组名和函数名，就达到同样效果。这种情况下也不需要变元R，仍然用E就够了。所以ppt上采用简化的实参形式。

函数返回语句

- $S \longrightarrow \text{return}\ E$ { S.code=E.code++gen[RETURN ?E.place]}

小结

- 掌握QTAC、QAST、了解逆波兰、LLVM-IR
- 为各语法单元生成中间代码（10.2-10.5）
- 以下属性名及各个属性含义
  - place; code; tc; fc
- 属性文法中用到的功能函数
  - newvar(); newl(); merg_code(); lookup(); update[]
  - gen[]; genl[]; emit[]; emit()
- 写属性文法与读属性文法；属性方程的程序实现

- 作业：习题10.2(1)、10.3、10.5

- 关于习题10.2：C语言中的for语句的中间代码生成。
- 面向主文法有：
  - $S \longrightarrow \text{for}(S;B;S)S$
  - 其意义如下：

```
S[1].code
while(B.code){
        S[3].code
        S[2].code
}
```

- 对QL文法可扩充：
  - $S \longrightarrow \text{for}(N;N;N)S;$
  - $N \longrightarrow E \mid \varepsilon$